

Ejercicios de Repaso

Programacion · Un ejercicio por cada Resultado de Aprendizaje (RA)

RA1	UT2 - Introduccion a la programacion
RA2	UT3 - Introduccion a POO
RA3	UT2 - Estructuras de control
RA4	UT3 - Clases y objetos
RA5	UT10 - Ficheros
RA6	UT6, UT9 - Arrays y colecciones
RA7	UT7 Excepciones + UT8 POO avanzada
RA8	Tema 11 - Bases de datos (JDBC)
RA9	Tema 11 - Bases de datos (JDBC)

9 ejercicios, uno por RA. Incluye un ejercicio de “¿qué se muestra por pantalla?”, uno de “mejora el código” y el de ficheros en versión sencilla. Las soluciones están al final del documento.

RA1*UT2 - Introduccion a la programacion*

Escribe un programa que pida por teclado el **precio** de un producto (double) y el **número de unidades** (int). El programa debe calcular el importe total, aplicarle un **21% de IVA** y mostrar por pantalla el precio final con un mensaje claro.

Usa Scanner para la entrada y variables del tipo adecuado.

Tu respuesta:

RA2*UT3 - Introduccion a la POO*

Crea una clase **Producto** con dos atributos: nombre (String) y precio (double). Añade un método mostrar() que imprima los dos atributos. En una clase con main, crea **dos** objetos Producto, asigna sus valores y llama a mostrar() en cada uno.

Tu respuesta:

RA3*UT2 - Estructuras de control | ¿Qué se muestra por pantalla?*

Analiza el siguiente código **sin ejecutarlo** y escribe la **salida exacta**, línea por línea, que aparece por pantalla:

```
int suma = 0;
for (int i = 1; i <= 6; i++) {
    if (i % 2 == 0) {
        suma += i;
    } else {
        suma -= 1;
    }
    System.out.println("i=" + i + " suma=" + suma);
}
System.out.println("Total: " + suma);
```

Salida exacta:

Amplia la clase **Producto** del ejercicio anterior:

- Añade un **constructor** que reciba nombre y precio.
- Haz los atributos **privados** y crea sus **getters y setters**.
- Sobrescribe `toString()` para devolver "Producto: nombre - X.XX €".
- Añade un método `aplicarDescuento(double pct)` que reste ese porcentaje al precio.

En el `main`: crea un `Producto` con el constructor, aplícale un 10% de descuento y muéstralo con `System.out.println(p)`.

Tu respuesta:

El fichero **Biblioteca.txt** tiene una línea por documento con este formato:

```
Titulo:Autor:Codigo:Anio:Estado:Edicion
```

Ejemplo de contenido real del fichero:

```
Cien Anios de Soledad:Garcia Marquez:9780307474728:1967:DISPONIBLE:1 edicion
National Geographic:Varios:12345678X:2020:PRESTADO:National Geographic
El Senor de los Anillos:Tolkien:9780618640157:1954:DISPONIBLE:2 edicion
...
```

Escribe un programa que **lea el fichero línea a línea** con `BufferedReader`, separe cada línea por : y muestre por pantalla **solo el título** de los documentos cuyo estado sea **DISPONIBLE**. No hay que ordenar ni escribir en ningún fichero: solo leer y filtrar.

Recuerda usar try-catch y cerrar el fichero.

Tu respuesta:

El siguiente programa debería contar cuántas veces aparece "Ana" en la lista, pero **no funciona**. Tiene **dos errores**: uno provoca una excepción en tiempo de ejecución y otro hace que el resultado sea incorrecto. **Encuétralos y reescribe el código corregido**.

```
ArrayList<String> nombres = new ArrayList<>();
nombres.add("Ana");
nombres.add("Pepe");
nombres.add("Ana");
nombres.add("Carlos");

int cont = 0;
for (int i = 0; i <= nombres.size(); i++) {
    if (nombres.get(i) == "Ana") {
        cont++;
    }
}
System.out.println("Ana aparece " + cont + " veces");
```

Errores encontrados y código corregido:

Crea una clase **abstracta** `Figura` con un método abstracto `double area()`. Crea dos clases hijas: `Circulo` (atributo `radio`) y `Rectangulo` (base y altura), cada una con su propio `area()`.

En el `main`, parte de este texto introducido por el usuario (simulado como `String`) para el radio:

```
String entrada = "hola"; // deberia ser un numero
```

Convierte la entrada a `double` con `Double.parseDouble(entrada)` dentro de un `try-catch` que capture `NumberFormatException` y muestre el mensaje *"Radio no válido"*. Si la conversión es correcta, crea un `Circulo` y muestra su área.

Tu respuesta:

Dada una base de datos MySQL llamada **instituto** con una tabla `alumnos(id, nombre, nota)`, escribe un programa Java que se **conecte** a la base de datos y muestre por pantalla el **nombre y la nota** de todos los alumnos.

Apoyate en este esqueleto de conexión (rellena los puntos suspensivos):

```
String url = "jdbc:mysql://localhost:3306/instituto";
Connection con = DriverManager.getConnection(url, "root", "1234");
// ... crea un Statement, ejecuta el SELECT,
// recorre el ResultSet y muestra nombre y nota
```

Recuerda gestionar `SQLException` con `try-catch` y cerrar la conexión.

Tu respuesta:

Sobre la misma tabla `alumnos(id, nombre, nota)`, escribe un programa que **inserte un nuevo alumno** en la base de datos usando un `PreparedStatement` con parámetros (?), no concatenando la cadena SQL.

El nombre y la nota del nuevo alumno los puedes fijar en variables (por ejemplo "Lucia" y 8.5). Tras ejecutar el `INSERT`, muestra cuántas filas se han insertado y gestiona `SQLException`.

Pista sobre la estructura:

```
String sql = "INSERT INTO alumnos (nombre, nota) VALUES (?, ?)";
PreparedStatement ps = con.prepareStatement(sql);
// ps.setString(1, ...); ps.setDouble(2, ...);
// int filas = ps.executeUpdate();
```

Tu respuesta:

Soluciones

RA1 - Precio con IVA

```
import java.util.Scanner;
public class PrecioIVA {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Precio: ");
        double precio = sc.nextDouble();
        System.out.print("Unidades: ");
        int unidades = sc.nextInt();
        double total = precio * unidades;
        double conIva = total * 1.21;
        System.out.println("Precio final (IVA incl.): " + conIva + " euros");
        sc.close();
    }
}
```

RA2 - Clase Producto basica

```
public class Producto {
    String nombre;
    double precio;
    void mostrar() {
        System.out.println(nombre + " -> " + precio + " euros");
    }
}

public class Main {
    public static void main(String[] args) {
        Producto p1 = new Producto();
        p1.nombre = "Boli"; p1.precio = 1.20;
        Producto p2 = new Producto();
        p2.nombre = "Cuaderno"; p2.precio = 3.50;
        p1.mostrar();
        p2.mostrar();
    }
}
```

RA3 - Que se muestra por pantalla

Se va acumulando: en los **impares** resta 1 y en los **pares** suma el valor de i.

```
i=1 suma=-1
i=2 suma=1
i=3 suma=0
i=4 suma=4
i=5 suma=3
i=6 suma=9
Total: 9
```

RA4 - Producto con constructor, encapsulacion y toString


```

public class Producto {
    private String nombre;
    private double precio;
    public Producto(String nombre, double precio) {
        this.nombre = nombre; this.precio = precio;
    }
    public String getNombre() { return nombre; }
    public void setNombre(String n) { this.nombre = n; }
    public double getPrecio() { return precio; }
    public void setPrecio(double p) { this.precio = p; }
    public void aplicarDescuento(double pct) {
        precio = precio - (precio * pct / 100);
    }
    @Override
    public String toString() {
        return "Producto: " + nombre + " - " + precio + " euros";
    }
    public static void main(String[] args) {
        Producto p = new Producto("Cuaderno", 3.50);
        p.aplicarDescuento(10);
        System.out.println(p);
    }
}

```

RA5 - Ficheros: títulos DISPONIBLES (facil)

```

import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;
public class LeerBiblioteca {
    public static void main(String[] args) {
        try (BufferedReader br = new BufferedReader(
            new FileReader("Biblioteca.txt"))) {
            String linea;
            while ((linea = br.readLine()) != null) {
                String[] partes = linea.split(":");
                if (partes[4].equals("DISPONIBLE")) {
                    System.out.println(partes[0]); // titulo
                }
            }
        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}

```

RA6 - Mejora el código (colecciones)

Error 1: `i <= nombres.size()` se sale del array (`IndexOutOfBoundsException`). Debe ser `i < nombres.size()`.

Error 2: `==` compara referencias, no contenido. Para `String` se usa `.equals()`.

```

for (int i = 0; i < nombres.size(); i++) {
    if (nombres.get(i).equals("Ana")) {
        cont++;
    }
}
// Salida: Ana aparece 2 veces

```

RA7 - Abstracta + herencia + excepcion

```

abstract class Figura {
    abstract double area();
}
class Circulo extends Figura {
    double radio;
    Circulo(double radio) { this.radio = radio; }
    double area() { return Math.PI * radio * radio; }
}
class Rectangulo extends Figura {
    double base, altura;
    Rectangulo(double b, double h) { base = b; altura = h; }
    double area() { return base * altura; }
}
public class Main {
    public static void main(String[] args) {
        String entrada = "hola";
        try {
            double r = Double.parseDouble(entrada);
            Circulo c = new Circulo(r);
            System.out.println("Area: " + c.area());
        } catch (NumberFormatException e) {
            System.out.println("Radio no valido");
        }
    }
}

```

RA8 - JDBC SELECT

```

import java.sql.*;
public class ConsultaAlumnos {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/instituto";
        try (Connection con = DriverManager.getConnection(url, "root", "1234");
            Statement st = con.createStatement();
            ResultSet rs = st.executeQuery(
                "SELECT nombre, nota FROM alumnos")) {
            while (rs.next()) {
                System.out.println(rs.getString("nombre")
                    + " -> " + rs.getDouble("nota"));
            }
        } catch (SQLException e) {
            System.out.println("Error BD: " + e.getMessage());
        }
    }
}

```

RA9 - JDBC INSERT con PreparedStatement

```
import java.sql.*;
public class InsertarAlumno {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/instituto";
        String sql = "INSERT INTO alumnos (nombre, nota) VALUES (?, ?)";
        try (Connection con = DriverManager.getConnection(url, "root", "1234");
            PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setString(1, "Lucia");
            ps.setDouble(2, 8.5);
            int filas = ps.executeUpdate();
            System.out.println("Filas insertadas: " + filas);
        } catch (SQLException e) {
            System.out.println("Error BD: " + e.getMessage());
        }
    }
}
```